

LAPORAN TUGAS AKHIR MATAKULIAH
WEB SERVICE PROGRAMMING
SEMESTER GENAP 2025/2026



Pengujian API – SaleNote - Sistem Pencatatan Penjualan Toko

Disusun oleh:

Muhammad Rifqii	(2418045)
Kosmas Anyeq Beraan	(2418057)
David Andrew Gregorius Tingang	(2418058)
Petrus Aldinando Ndu Ea	(2418049)

PROGRAM STUDI TEKNIK INFORMATIKA S-1
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI NASIONAL MALANG
2025

I. Link GitHub Project:

<https://github.com/MuhammadRifqii10/SaleNote---Sistem-Pencatatan-Penjualan-Toko.git>

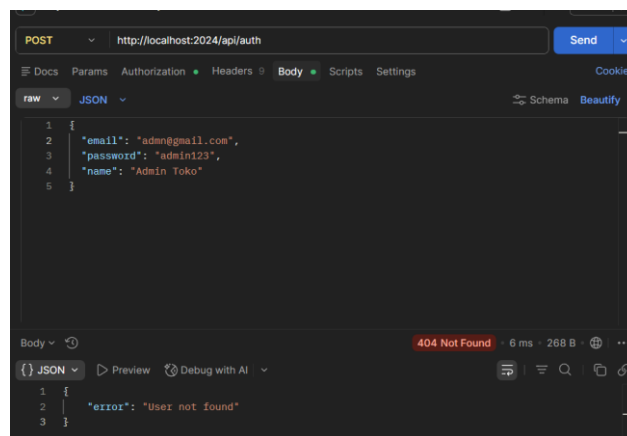
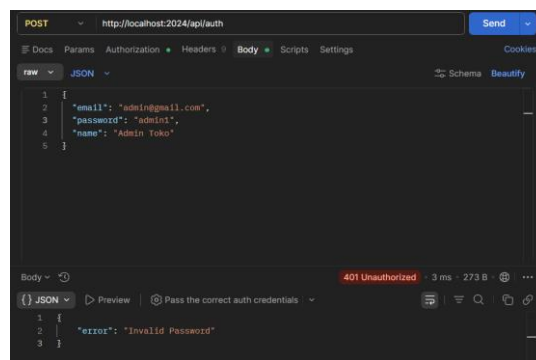
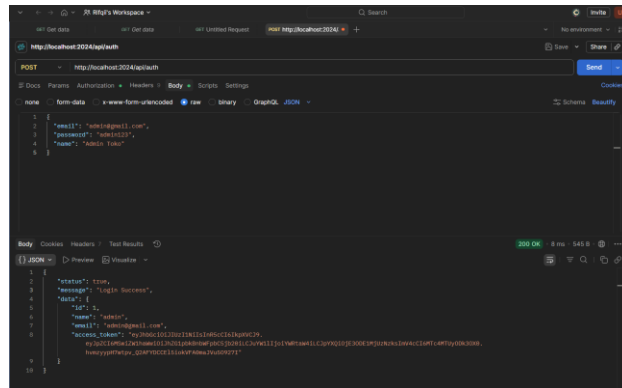
II. End Point

Berikut adalah daftar Endpoint API yang di implementasikan pada sistem manajemen dan pencatatan produk jam tangan mewah:

No	Endpoint	Method	Fungsi
1	/api/auth	Post	Mengirim kredensial admin untuk memverifikasi autentikasi dan menghasilkan token akses JWT.
2	/api/categories	Post	Menambahkan kategori menu baru (Coffee, Pastry, Non-Coffee) ke dalam database
3	/api/products	Post	Menambahkan data menu produk baru dan menghubungkannya dengan ID kategori terkait.
4	/api/products	Get	untuk mengambil data semua pengguna atau mengelola data pengguna khusus admin
5	/api/customers	Post	Mendaftarkan data pelanggan baru ke dalam sistem untuk pencatatan transaksi.
6	/api/sales/	Post	Mencatat transaksi penjualan baru, memotong stok menu otomatis, dan menghitung total harga.
7	/api/sales/	Get	Untuk mengambil semua riwayat data laporan transaksi penjualan kasir yang tersimpan di database.

III. Dokumentasi End Point

1. /POST /api/auth



2. POST /api/categories

A screenshot of a REST client interface. The top bar shows a POST request to `http://localhost:2024/api/categories/`. The request body is a JSON object: `{ "name": "Coffee" }`. The response status is `201 Created` with a response time of 12 ms and a body size of 333 B. The response body is a JSON object: `{ "status": true, "message": "Data kategori berhasil ditambahkan", "data": { "id": 8, "name": "Coffee" } }`.

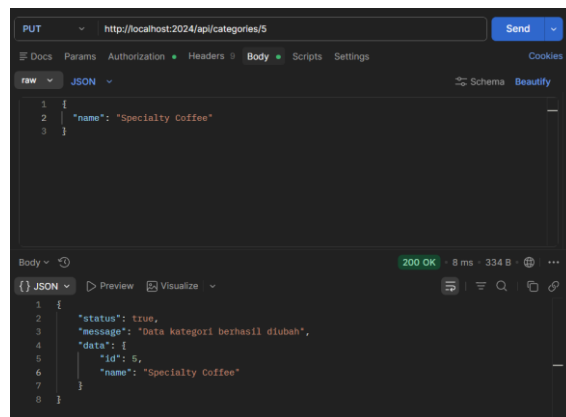
A screenshot of a REST client interface. The top bar shows a POST request to `http://localhost:2024/api/categories/`. The request body is a JSON object: `{ "name": "Coffee" }`. The response status is `409 Conflict` with a response time of 10 ms and a body size of 302 B. The response body is a JSON object: `{ "status": "failed", "errors": [ { "category name already exists" } ] }`.

A screenshot of a REST client interface. The top bar shows a POST request to `http://localhost:2024/api/categories/`. The request body is a JSON object: `{ "name": "Coffee" }`. The response status is `401 Unauthorized` with a response time of 4 ms and a body size of 270 B. The response body is a JSON object: `{ "error": "Invalid Token" }`.

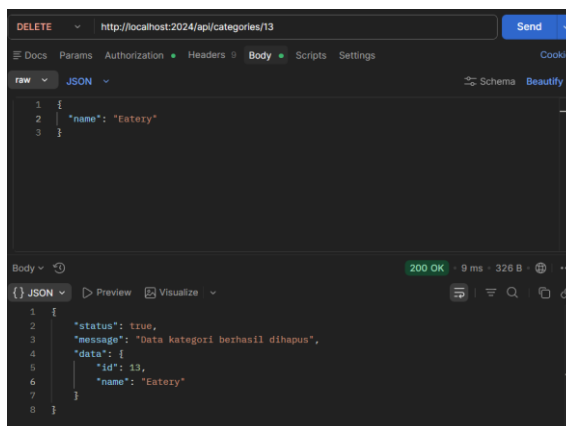
3. Get /api/categories

A screenshot of a REST client interface. The top bar shows a GET request to `http://localhost:2024/api/categories`. The response status is `200 OK` with a response time of 6 ms and a body size of 611 B. The response body is a JSON object: `{ "status": true, "message": "Data kategori ditemukan", "data": [{ "id": 8, "name": "Coffee", "createdAt": "2026-06-15T12:17:57.983Z", "updatedAt": "2026-06-15T12:17:57.983Z" }, { "id": 11, "name": "Non Coffee", "createdAt": "2026-06-15T12:29:37.486Z", "updatedAt": "2026-06-15T12:29:37.486Z" }, { "id": 12, "name": "Pastry", "createdAt": "2026-06-15T12:29:46.199Z", "updatedAt": "2026-06-15T12:29:46.199Z" }] }`.

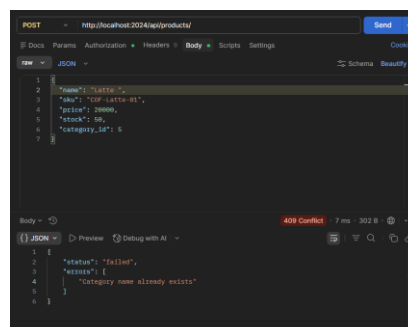
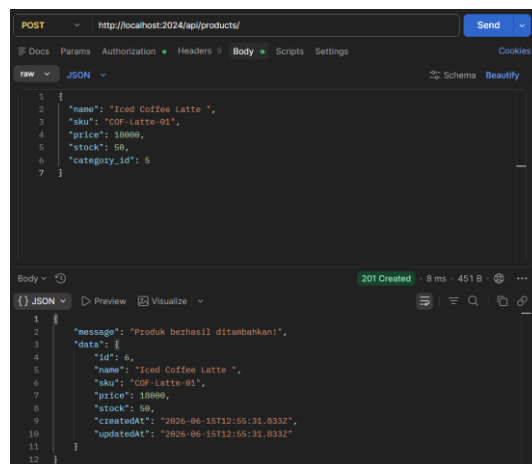
4. PUT /api/categories/:id



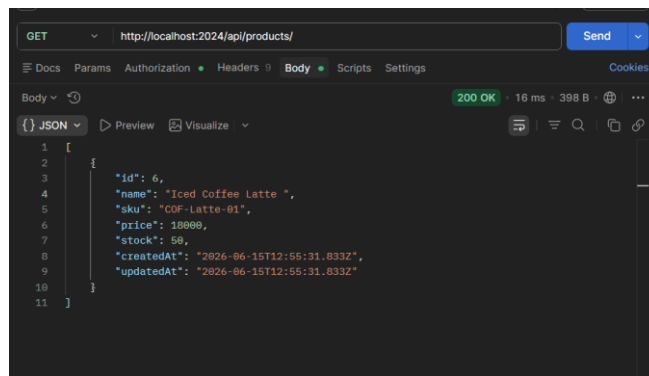
5. DELETE /api/categories/:id



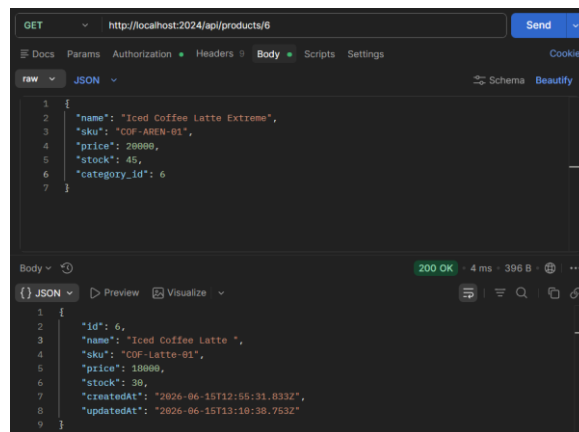
6. POST /api/products



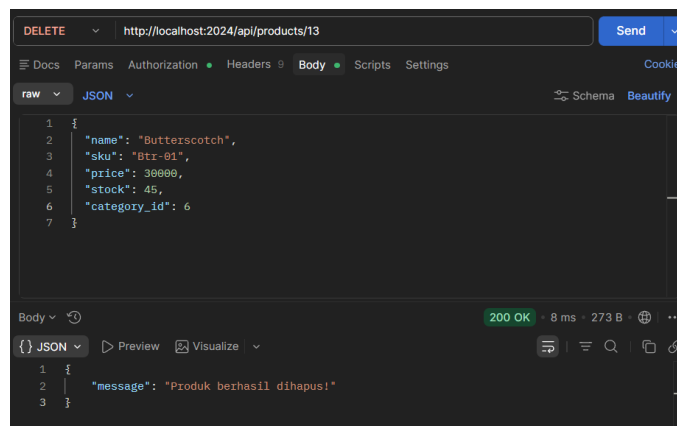
7. GET /api/products



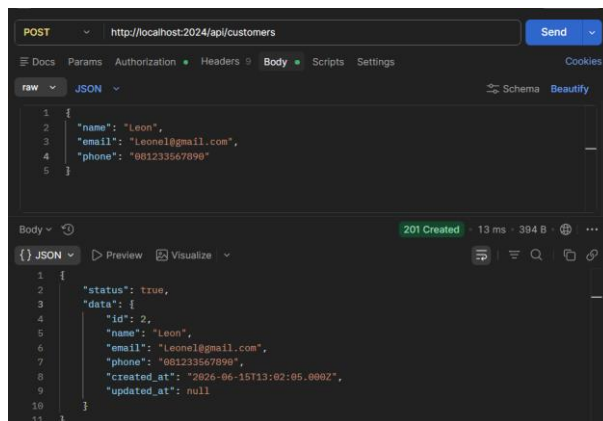
8. PUT /api/products/:id



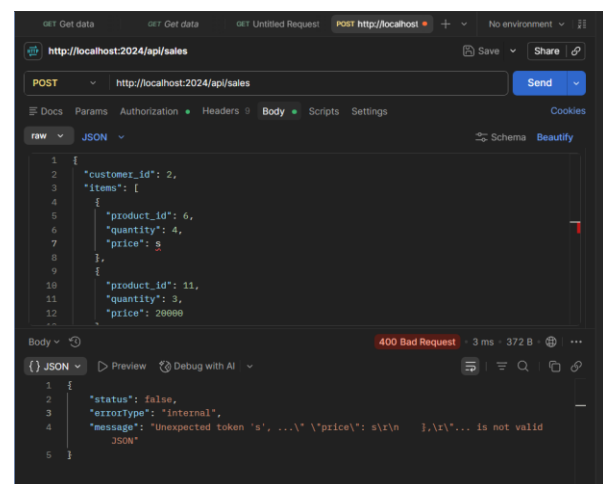
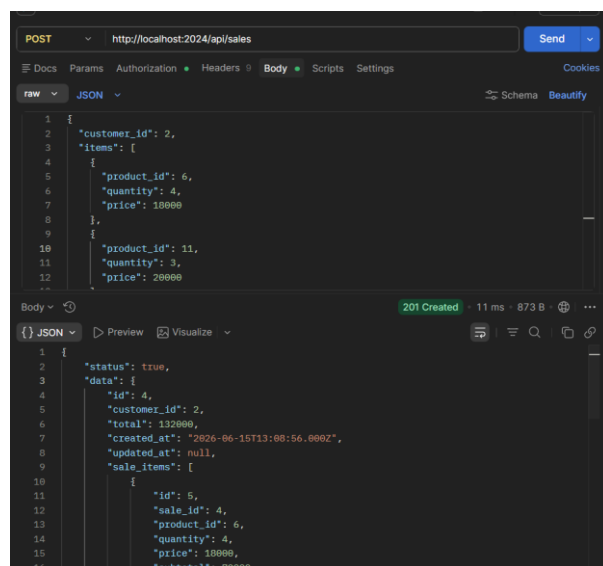
9. DELETE /api/products/:id



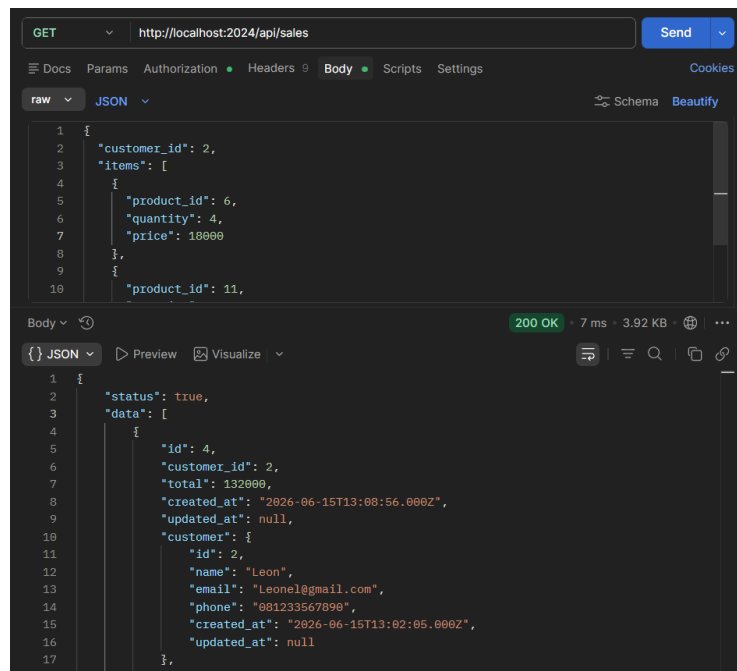
10. POST /api/customers



11. POST /api/sales



12. GET/api/sales



IV. Dokumentasi Schema

users X +				
Filters None Fields All Showing 1 of 1 Add record				
id #	name A	email A	email_verified_at ?	password A
1	admin	admin@gmail.com	null	admin123

```
model users {
  id          BigInt      @id @default(autoincrement())
  @db.UnsignedBigInt
  name        String      @db.VarChar(255)
  email       String      @unique(map: "users_email_unique")
  @db.VarChar(255)
  email_verified_at DateTime? @db.Timestamp(0)
  password    String      @db.VarChar(255)
  remember_token String?   @db.VarChar(100)
  created_at  DateTime?   @db.Timestamp(0)
  updated_at  DateTime?   @db.Timestamp(0)
  news        news[]
}
```

users X products X +							
Filters None Fields All Showing 3 of 3 Add record							
id #	name A	sku A	price #	stock #	createdAt	updatedAt	sale_items {}
6	Iced Coffee Latte	CD-Latte-01	18000	30	2026-06-15T12:55:31.833Z	2026-06-15T13:10:38.753Z	5 sale_items
11	Matcha Latte	Matc-01	20000	15	2026-06-15T13:05:15.134Z	2026-06-15T13:10:38.753Z	5 sale_items
12	Butter Croissant	Cro-01	25000	30	2026-06-15T13:06:38.326Z	2026-06-15T13:10:38.326Z	0 sale_items

```
model products {
  id          BigInt      @id @default(autoincrement())
  @db.UnsignedBigInt
  name        String      @db.VarChar(255)
  sku         String      @unique(map: "products_sku_unique")
  @db.VarChar(100)
  price       Float       @db.Float
  stock       Int         @db.Int
}
```



```

    createdAt    DateTime @default(now())
    updatedAt    DateTime @updatedAt
    sale_items  sale_items[]
}

```

	users	products	categories	+
	Filters	None	Fields	All
	Showing	3 of 3	Add record	
id #	name A	createdAt	updatedAt	news []
5	Specialty Coffee	2026-06-15T12:17:57.983Z	2026-06-15T13:27:36.262Z	0 news
11	Non Coffee	2026-06-15T12:29:37.480Z	2026-06-15T12:29:37.480Z	0 news
12	Pastry	2026-06-15T12:29:46.199Z	2026-06-15T12:29:46.199Z	0 news

```

model categories {
  id          BigInt      @id @default(autoincrement())
  @db.UnsignedBigInt
  name        String      @unique(map: "categories_name_unique")
  @db.VarChar(100)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  news        news[]
}

```

Filters

None

Fields

All

Showing 5 of 5

Add record

	id #	customer_id #?	total #	created_at ?	updated_at ?	customer ()?	sale_items []
	4	2	132000	2026-06-15T13:08:56.000Z	null	customers	2 sale_items
	5	2	92000	2026-06-15T13:09:55.000Z	null	customers	2 sale_items
	6	2	132000	2026-06-15T13:10:01.000Z	null	customers	2 sale_items
	7	2	60000	2026-06-15T13:10:31.000Z	null	customers	2 sale_items
	8	2	132000	2026-06-15T13:10:39.000Z	null	customers	2 sale_items

```

model sales {
  id          BigInt      @id @default(autoincrement())
  @db.UnsignedBigInt
  customer_id BigInt?     @db.UnsignedBigInt
  total        Float      @db.Float
  created_at   DateTime?  @db.Timestamp(0)
  updated_at   DateTime?  @db.Timestamp(0)

  customer     customers? @relation(fields: [customer_id],
  references: [id], onDelete: SetNull, onUpdate: Restrict, map:
  "sales_customer_id_foreign")
  sale_items  sale_items[]

  @@index([customer_id], map: "sales_customer_id_foreign")
}

```

	Filters	None	Fields	All	Showing	10 of 10	Add record
id #	sale_id #	product_id #	quantity #	price #	subtotal #	sales ()	products ()
5	4	6	4	18000	72000	sales	products
6	4	11	3	20000	60000	sales	products
7	5	6	4	8000	32000	sales	products
8	5	11	3	20000	60000	sales	products
9	6	6	4	18000	72000	sales	products
10	6	11	3	20000	60000	sales	products
11	7	6	4	0	0	sales	products
12	7	11	3	20000	60000	sales	products
13	8	6	4	18000	72000	sales	products
14	8	11	3	20000	60000	sales	products

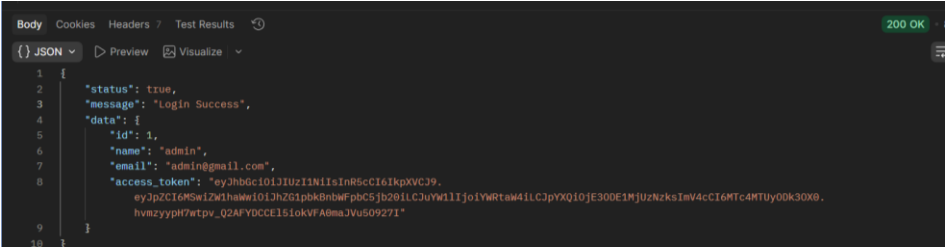
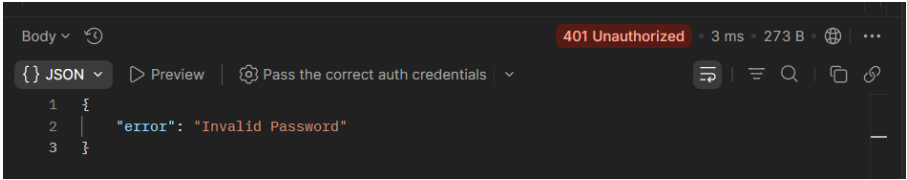
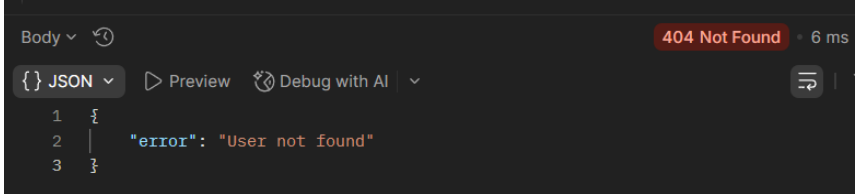
```
model sale_items {
    id          BigInt @id @default(autoincrement())
    @db.UnsignedBigInt
    sale_id     BigInt @db.UnsignedBigInt
    product_id  BigInt @db.UnsignedBigInt
    quantity    Int     @db.Int
    price       Float   @db.Float
    subtotal    Float   @db.Float

    sales       sales    @relation(fields: [sale_id], references: [id],
onDelete: Cascade, onUpdate: Restrict, map:
"sale_items_sale_id_foreign")
    products    products @relation(fields: [product_id], references:
[id], onDelete: Restrict, onUpdate: Restrict, map:
"sale_items_product_id_foreign")

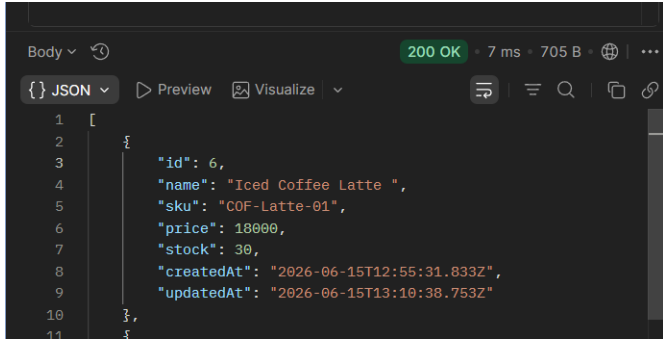
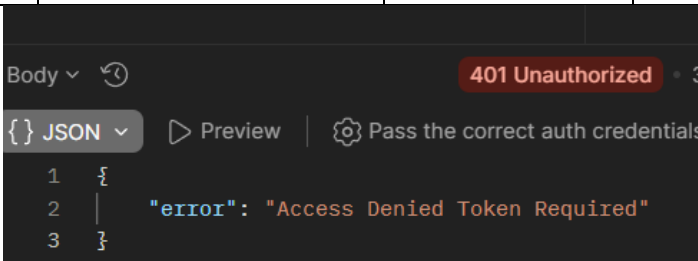
    @@index([sale_id], map: "sale_items_sale_id_foreign")
    @@index([product_id], map: "sale_items_product_id_foreign")
}
```

V. Pengujian dan Hasil Pengujian

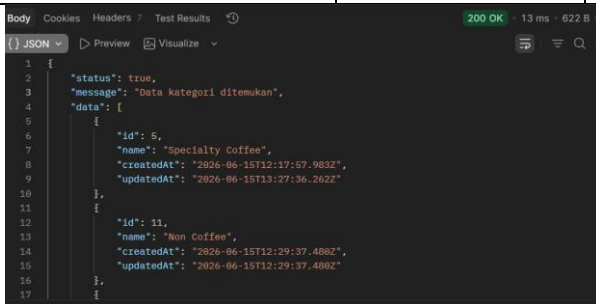
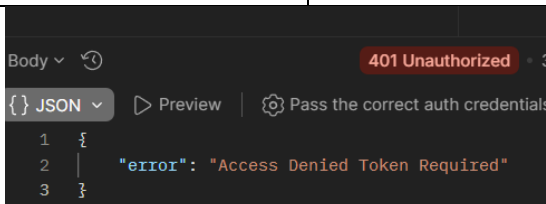
1. Post/auth/

Kasus Uji	Input	Expected Status	Validasi
Login dengan kredensial benar	Username, email, password	200 OK	Berhasil mendapatkan token akses JWT.
			
Login dengan password salah	Username, email, password salah	401 Unauthorized	Error: Invalid Password
			
Login dengan user tidak terdaftar	Username, Email, Tidak ditemukan	404, Not Found	Error: User not found
			
Password kosong	Username, Email	400, "Unexpected token"	Error: Bad Request
			

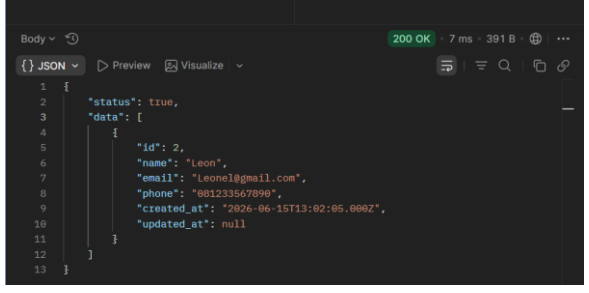
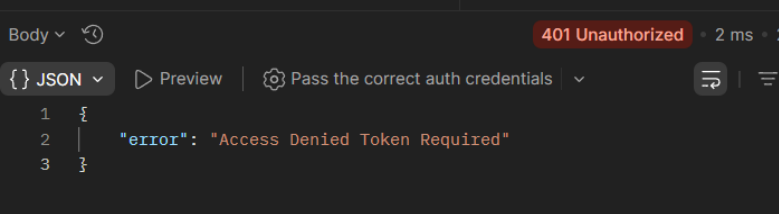
2. Get/product

Kasus Uji	Input	Expected Status	Validasi
Ketika akses dengan token	-	200OK	Product retrieved successfully
 <pre> 1 [2 { 3 "id": 6, 4 "name": "Iced Coffee Latte ", 5 "sku": "COF-Latte-01", 6 "price": 18000, 7 "stock": 30, 8 "createdAt": "2026-06-15T12:55:31.833Z", 9 "updatedAt": "2026-06-15T13:10:38.753Z" 10 }, 11] </pre>			
Ketika token invalid atau tanpa token	-	401,Error message	Unauthorized
 <pre> 1 { 2 "error": "Access Denied Token Required" 3 } </pre>			

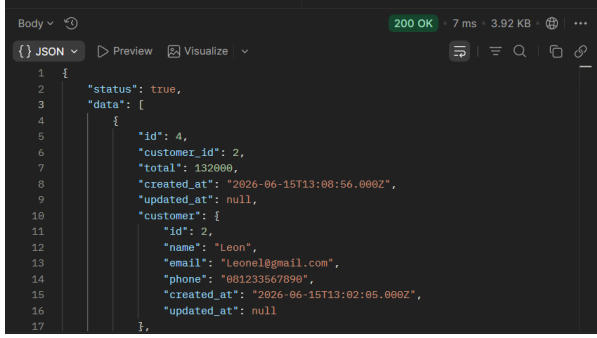
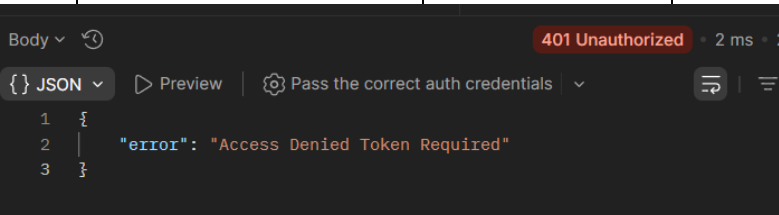
3. Get/categories

Kasus Uji	Input	Expected Status	Validasi
Ketika akses dengan token yang valid	-	200 OK	Data kategori berhasil ditemukan
 <pre> 1 { 2 "status": true, 3 "message": "Data kategori ditemukan", 4 "data": [5 { 6 "id": 5, 7 "name": "Specialty Coffee", 8 "createdAt": "2026-06-15T12:17:57.983Z", 9 "updatedAt": "2026-06-15T13:27:36.262Z" 10 }, 11 { 12 "id": 11, 13 "name": "Non Coffee", 14 "createdAt": "2026-06-15T12:29:37.486Z", 15 "updatedAt": "2026-06-15T12:29:37.486Z" 16 } 17] 18 } </pre>			
Ketika token invalid atau tanpa token	-	401, Error message	Unauthorized
 <pre> 1 { 2 "error": "Access Denied Token Required" 3 } </pre>			

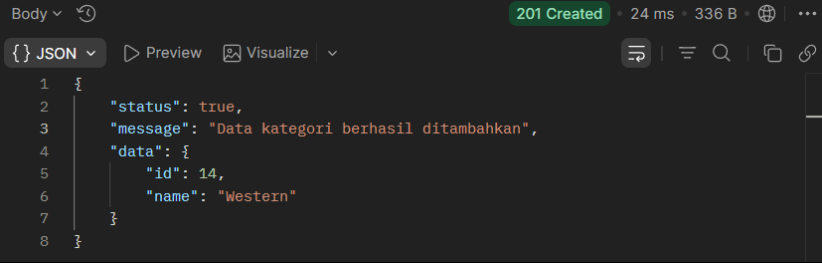
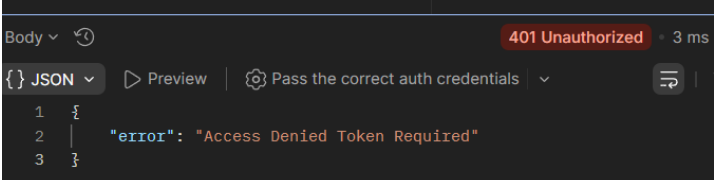
4. Get/customers

Kasus Uji	Input	Expected Status	Validasi
Ketika mengambil daftar customer	-	200, menampilkan data customer	status : true
 <pre> 1 { 2 "status": true, 3 "data": [4 { 5 "id": 2, 6 "name": "Leon", 7 "email": "Leone1@gmail.com", 8 "phone": "081233567890", 9 "created_at": "2026-06-15T13:02:05.000Z", 10 "updated_at": null 11 } 12] 13 } </pre>			
Ketika Tanpa Token	-	401, Error message	Unauthorized
 <pre> 1 { 2 "error": "Access Denied Token Required" 3 } </pre>			

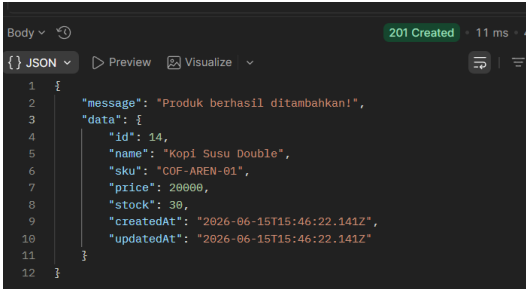
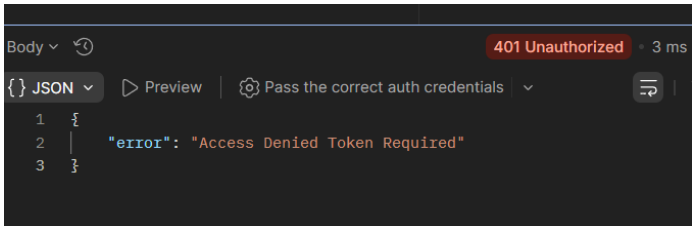
5. Get/sales

Kasus Uji	Input	Expected Status	Validasi
Ketika mengambil daftar penjualan	-	200, menampilkan data penjualan	status : true
 <pre> 1 { 2 "status": true, 3 "data": [4 { 5 "id": 4, 6 "customer_id": 2, 7 "total": 132000, 8 "created_at": "2026-06-15T13:08:56.000Z", 9 "updated_at": null, 10 "customer": { 11 "id": 2, 12 "name": "Leon", 13 "email": "Leone1@gmail.com", 14 "phone": "081233567890", 15 "created_at": "2026-06-15T13:02:05.000Z", 16 "updated_at": null 17 } 18 } 19] 20 } </pre>			
Ketika Tanpa Token	-	401, Error message	Unauthorized
 <pre> 1 { 2 "error": "Access Denied Token Required" 3 } </pre>			

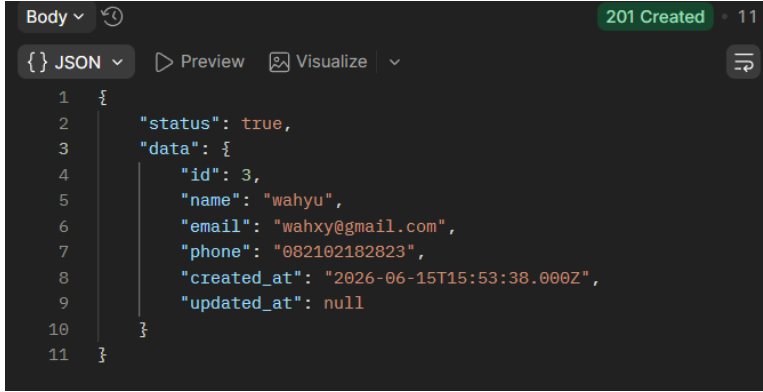
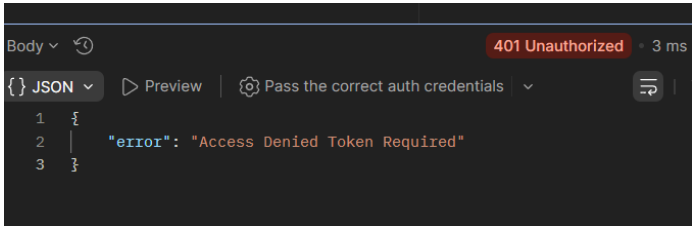
6. Post/categoris

Kasus Uji	Input	Expected Status	Validasi
Ketika tambah kategori dengan data yang valid	Nama kategori	201 Created menampilkan data kategori	Data kategori berhasil ditambahkan
			
Ketika tanpa ada token	-	401, Error message	Unauthorized
			

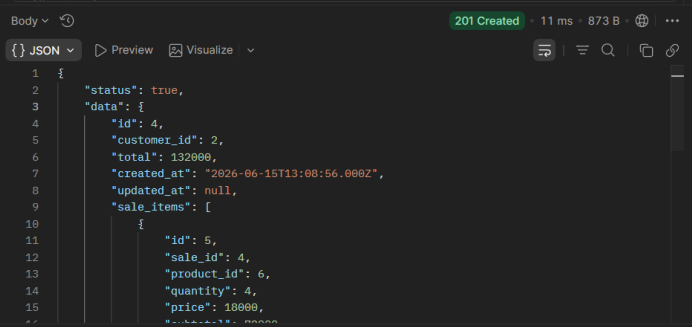
7. Post/Product

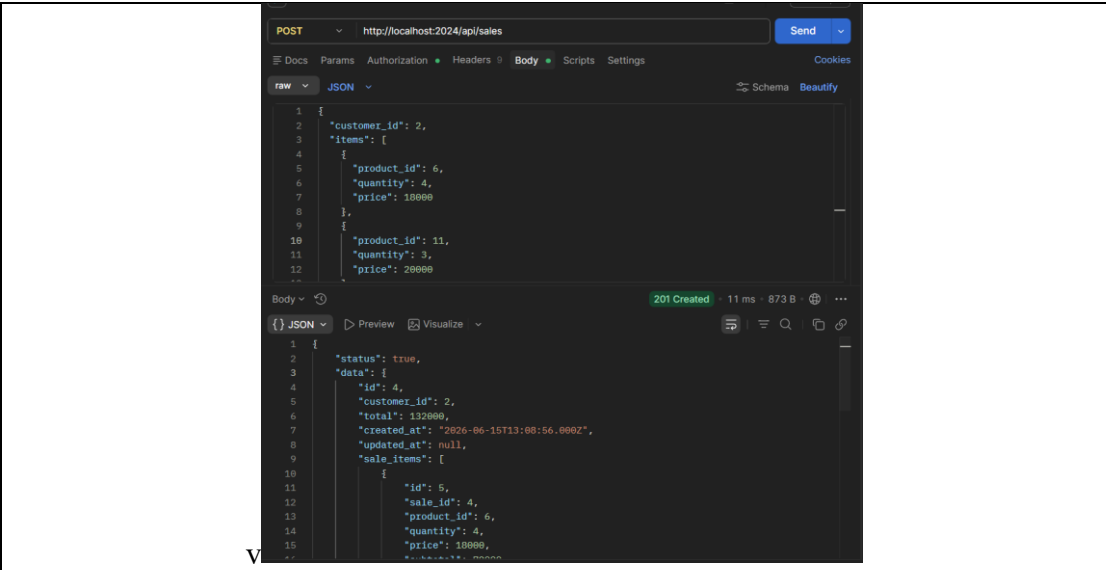
Kasus Uji	Input	Expected Status	Validasi
Ketika tambah produk dengan data yang valid	Nama produk, sku, price, stock, category id	201 Created menampilkan data produk	Data produk berhasil ditambahkan
			
Ketika tanpa ada token	-	401, Error message	Unauthorized
			

8. Post/Customers

Kasus Uji	Input	Expected Status	Validasi
Ketika tambah customer dengan data yang valid	Name, email, phone	201 Created menampilkan data customer	True
			
Ketika tanpa ada token	-	401, Error message	Unauthorized
			

9. Post/Sales

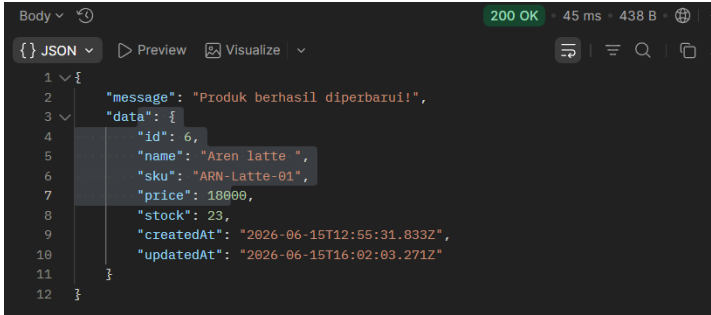
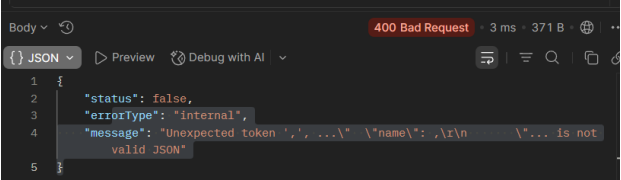
Kasus Uji	Input	Expected Status	Validation
Ketika menambahkan id sales	Customer id, produk id, quantity, price	201, data sales berhasil ditambahkan	true
			
Ketika memasukkan id yang tidak terdaftar	Selain 1 dan 3	404, Vehicle not found	Error: Not Found



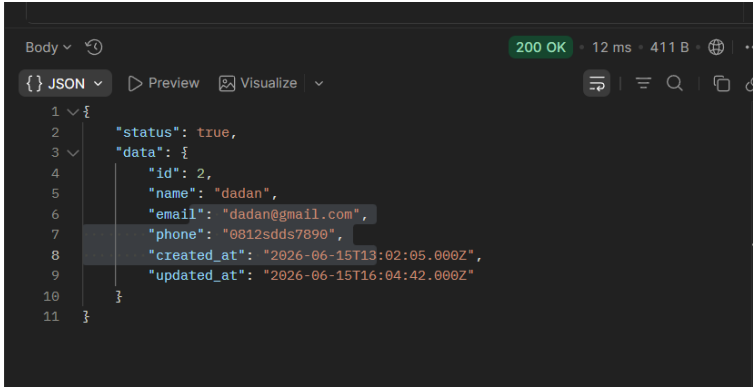
10. PUT/categories/:id

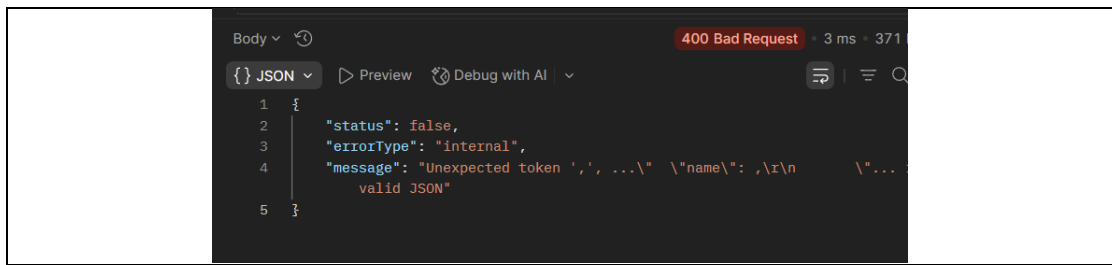
Kasus Uji	Input	Expected Stasus	Validation
Ketika mengupdate data kategori by id yang sudah terdaftar	Data kategori baru	200, Menampilkan data yang telah diubah	Data berhasil diubah
Ketika mengupdate data kosong	-	404, bad request	false

11. PUT/Product/:id

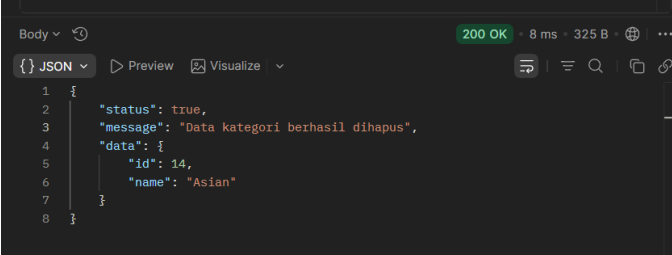
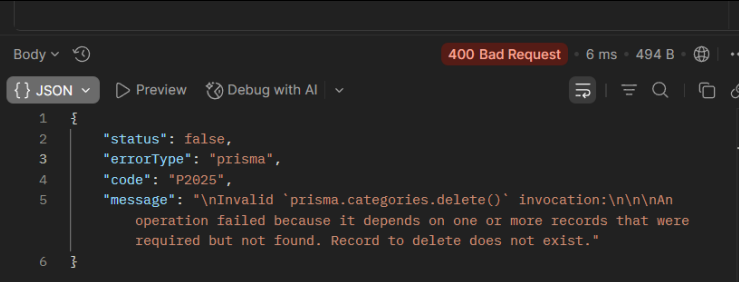
Kasus Uji	Input	Expected Status	Validation
Ketika mengupdate data product by id yang sudah terdaftar	Data produk baru	200, Menampilkan data yang telah diubah	Data berhasil diubah
 <pre> 1 { 2 "message": "Produk berhasil diperbarui!", 3 "data": { 4 "id": 6, 5 "name": "Aren latte ", 6 "sku": "ARN-Latte-01", 7 "price": 18000, 8 "stock": 23, 9 "createdAt": "2026-06-15T12:55:31.833Z", 10 "updatedAt": "2026-06-15T16:02:03.271Z" 11 } 12 } </pre>			
Ketika mengupdate data kosong	-	400, bad request	false
 <pre> 1 { 2 "status": false, 3 "errorType": "Internal", 4 "message": "Unexpected token ',', ...\" \\\"name\\\": \\\"r\\n\\\" \\\"... is not valid JSON\" 5 } </pre>			

12. Put/customers/:id

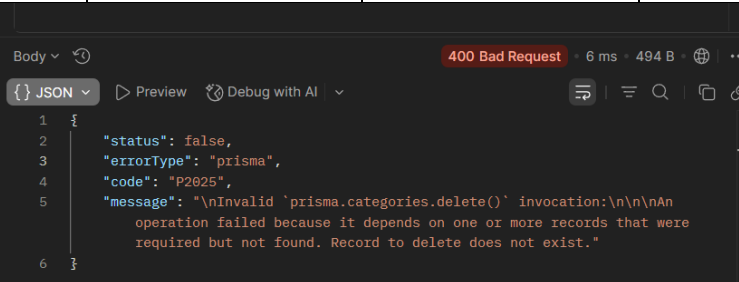
Kasus Uji	Input	Expected Status	Validation
Ketika mengupdate data customer by id yang sudah terdaftar	Data customer baru	200, Menampilkan data yang telah diubah	True
 <pre> 1 { 2 "status": true, 3 "data": { 4 "id": 2, 5 "name": "dadad", 6 "email": "dadad@gmail.com", 7 "phone": "0812sdds7890", 8 "created_at": "2026-06-15T13:02:05.000Z", 9 "updated_at": "2026-06-15T16:04:42.000Z" 10 } 11 } </pre>			
Ketika mengupdate data kosong	-	400, bad request	false



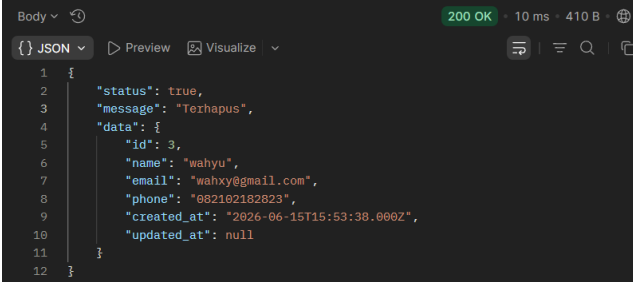
13. Delete/categories/:id

Kasus Uji	Input	Expected Status	Validation
Ketika menghapus data kategori	-	200, Data berhasil dihapus	True
			
Ketika akan menghapus id kategori yang tidak ada	-	400 bad request	false
			

14. Delete/product/:id

Kasus Uji	Input	Expected Status	Validation
Ketika menghapus data produk	-	200, Data berhasil dihapus	True
Ketika akan menghapus id produk yang tidak ada	-	400 bad request	false
			

15. Delete/customers/:id

Kasus Uji	Input	Expected Status	Validation
Ketika menghapus data customer	Id customer	200 OK	Terhapus
			
Ketika akan menghapus id customer yang tidak ada	-	400 bad request	false
	